

Building TinyCodeRAG Step-by-Step: A Lightweight Code Knowledge Base Solution

Share:    

In the previous article, we broke down the core components of RAG systems. Today, let's do something even cooler—personally build a TinyCodeRAG optimized specifically for code!

💡 Quick primer: RAG (Retrieval-Augmented Generation) technology effectively alleviates the "hallucination" problem of large models by combining external knowledge bases with AI generation capabilities.

You might wonder: "With TinyRAG already excellent, why reinvent the wheel?" Two reasons:

First, reinventing the wheel is the most solid learning path.

Second, in the process of reinventing the wheel, you can find ways to make it look better—a creative process that's truly refreshing.

Thus, [TinyCodeRAG](#) was born! It brings four core upgrades: ✅ Intelligent code chunking: specifically parses code data structures to build precise vector sets ✅ Ready to use: provides test API keys (I'll renew them periodically after use) ✅ Modular testing: each component has independent test cases ✅ Optimized conversation experience: full support for multi-turn contextual conversations

Less talk, let's get moving!

0. Project File Overview

TinyCodeRAG

```
├─ RAG
|   ├── embeddings.py      # Vectorization functionality wrapper
|   ├── chunker_text.py    # General text splitter
|   ├── chunker_code.py    # Dedicated code splitter 📌 Key innovation!
|   ├── vector_base.py     # Lightweight vector database
|   ├── llm.py             # LLM interface wrapper
|   ├── test_*.py          # Module test scripts (4 total)
|   └── tiny_code_rag.py   # RAG system integration entry
```

Below, we'll cover vectorization, text and code splitting, vector database implementation, LLM wrapping, and TinyCodeRAG integration application. Each module has corresponding test files for easy understanding.

1. Vectorization Engine

We first implement the core foundation of the RAG system: vectorization processing. In the `embeddings.py` file, we define a specialized class primarily responsible for two key functions:

1. **get_embeddings** : Converts text (or code snippets) into corresponding vector representations.
2. **cosine_similarity** : Calculates cosine similarity scores between two vectors.

Vector Generation Engine: OpenAI We chose OpenAI's `text-embedding-3-small` model to drive vector generation. This model is very flexible, capable of efficiently converting input text of **any length** (even super long) into a fixed-length **1536-dimensional vector**. This lays a solid foundation for subsequent similarity calculations.

Validation: Beyond Conversion, It's About Results To ensure this vectorization logic truly works, we designed test cases, focusing on verifying two capabilities:

Similarity discrimination effectiveness: Can the system accurately distinguish similarity levels between different texts? We tested with four key text segments:

```
# Control text
test_text_1 = "Hello, world! This is a test."

# Highly relevant to test_text_1
test_text_2 = "Hello, world! This is a embedding test."

# Different topic from test_text_1
test_text_3 = "I want to study how to use the embedding model."

# Super long text test
test_text_long = "... repeat long text ..." * 100
```

Test results met expectations:

```
test_text_1 and test_text_2 highly relevant => Similarity ≈ 0.8
test_text_1 and test_text_3 different topics => Similarity ≈ 0.1
test_text_1 and test_text_long also significantly different => Similarity ≈ 0.1
```

This validates that our vectorization and similarity calculation are reliable and effective in identifying semantic associations.

2. Text Splitting Module

When building RAG (Retrieval-Augmented Generation) systems, **chunking** is a critical component ensuring system performance, with its core being splitting text into appropriately sized and semantically complete segments.

Example usage:

```
# Read documents from specified path, max chunk length 600, 150 character
docs = ReadFiles('~/.workspace/tiny-universe').get_content(max_token_len=
```

Furthermore, to adapt to the special structure of codebases, we added a **dedicated code splitter** (`chunker_code.py`). Unlike general text chunking, this module specifically optimizes code processing logic:

1. **Function integrity preservation:** Prioritizes keeping code from the same function in the same chunk
2. **Smart file filtering:** Automatically identifies and processes source code files by extension
3. **Directory-level processing:** Supports direct input of folder paths for batch processing

```
# Split source code from specified directory, 150 character overlap betw
code_docs = split_to_segment("~/.workspace/tiny-universe", cover_content=
```

Parameter说明:

`cover_content` : Defines the number of overlapping characters between adjacent chunks, enhancing contextual semantic coherence (can be set to 0 in actual applications)

Path compatibility: Supports Mac system tilde `~` paths; Windows users need to replace with absolute paths

Through `test_chunker.py` , we performed chunking tests on the `tiny-universe` project. You can view the chunked text content.

3. Vector Database Module

Although current vector database options are abundant, complete deployment solutions involve high operational costs and don't facilitate understanding the core operational logic of RAG systems. To address this, we implemented a lightweight vector storage module based on the TinyRAG project.

```
class VectorStore:
    # Initialize text storage container
    def __init__(self, document: List[str] = ['']) -> None:

    # Call embedding model to convert text to vectors (note token consum
    def get_vector(self, EmbeddingModel: BaseEmbeddings) -> List[List[fl

    # Vector data persistence storage
    def persist(self, path: str = 'storage'):

    # Load preprocessed vectors from storage path
    def load_vector(self, path: str = 'storage') -> bool:

    # Calculate similarity between two vectors
    def get_similarity(self, vector1: List[float], vector2: List[float])

    # Semantic retrieval: input query text, return k most relevant resul
    def query(self, query: str, EmbeddingModel: BaseEmbeddings, k: int =
```

Through `load_vector()` reusing pre-stored vector data, we avoid redundant calculations and significantly reduce token consumption. The `query()` function encapsulates the full workflow from text encoding to similarity matching.

As shown in the `test_vector_base.py` example, business integration takes just three steps: initialize document container → load/generate vectors → execute semantic query:

```
# Prioritize loading existing vector data, generate in real-time if
if not vector_store.load_vector():
    vector_store.get_vector(OpenAIEmbedding())
    vector_store.persist()

# Execute semantic retrieval and print results
for doc in vector_store.query("RAG 的组成部分是那些?", OpenAIEmbedding(
    print(doc)
    print("-"*100)
```

4. LLM Wrapper

We wrapped a concise LLM calling module based on OpenAI API (located in `llm.py`). To make it easy for everyone to test, the project comes integrated with a **free test API key** by default.

Considering cost constraints (token consumption), we currently limit use to the `Doubao-1.5-lite-32k` model. Also, let's discuss a key point about RAG systems: **bigger model size isn't always better**.

In reality, model parameter selection should depend more on your **specific requirement complexity**. If your logic design isn't inherently complex, using a large model might actually introduce more uncertainty—smaller models are sometimes more stable.

5. TinyCodeRAG System Integration

In the `tiny_code_rag.py` file, we encapsulate the TinyCodeRAG calling logic, directly implementing the core functionality of the RAG system. Let's see how to use it:

```
## 1. Prepare code corpus and text corpus
code_docs = split_to_segmenmt("~/workspace/tiny-universe", cover_con
text_docs = ReadFiles('~workspace/tiny-universe').get_content(max_t

# Merge code corpus and text corpus
doc_contents = [doc.content for doc in code_docs] + text_docs

vector_store = VectorStore(document=doc_contents)

# Vectorize and persist
if not vector_store.load_vector():
    vector_store.get_vector(OpenAIEmbedding())
    vector_store.persist()

# 2. Get LLM
model = DoubaoLiteModel()

# 3. Write user input, after LLM output, add to history data, and lo
history = []
while True:
    user_input = input("Please enter your question: ")
    contents = vector_store.query(user_input, OpenAIEmbedding(), 3)
    response = model.chat(user_input, history, "\n".join(contents))
    print("\n")
    history.append({'role': 'user', 'content': user_input})
    history.append({'role': 'assistant', 'content': response})
```

When you're ready with the RAG corpus database, this file can be run directly for multi-turn conversations. Each conversation includes historical data.

CodeMLS

[GitHub](#) [Columns](#) [Articles](#) [Guestbook](#)

This project primarily implements the core logic of RAG systems, including vectorization, text and code splitting, vector database implementation, LLM wrapping, and TinyCodeRAG integration application.

Through this project, we fully implemented:  Code-sensitive knowledge base construction 
Retrieval-generation closed-loop system  Extensible multi-turn conversation framework

Try it now: 

[TinyCodeRAG](#)

✨ Welcome to Star/Fork/Issue! Your feedback drives my continued optimization~

Loading comments...

Unless otherwise stated, all articles on this blog are licensed under [CC BY-NC-SA 4.0](#) . Please credit the source when reposting.

[GitHub](#)[↗] [Articles](#)→ [Columns](#)→